

Semana 28 - Aula 1

Tópico Principal da Aula: Integração e Testes de Serviços Externos

Subtítulo/Tema Específico: Testes de Integração com APIs

Código da aula: [SIS]ANO2C2B4S28A1

Objetivos da Aula:

- Implementar testes de integração para garantir a qualidade e a correta comunicação com **APIs externas**.
- Conhecer as principais ferramentas e técnicas para simular e validar cenários críticos de integração.
- Interpretar os resultados dos testes para assegurar a confiabilidade e segurança das chamadas externas.

Recursos Adicionais (Sugestão, pode ser adaptado):

- Caderno para anotações;
- Acesso ao laboratório de informática e/ou internet.
- Ferramentas de teste de API (Postman, Insomnia, ou frameworks como Rest-Assured/Chai).

Exposição do Conteúdo

Referência do Slide: Tema 1 - Escolha de Ferramentas e Cenários

- **Definição:** A escolha da **ferramenta de teste de integração** e a **definição dos cenários** são os passos iniciais para garantir que a comunicação do sistema com APIs externas seja robusta. O foco é simular o comportamento real do usuário e os fluxos de dados entre sistemas.
- **Aprofundamento/Complemento (se necessário):** O teste de integração não se limita a verificar se a API externa está "no ar" (disponível), mas sim se a **lógica de negócio** da aplicação, que depende da API, funciona corretamente. Isso inclui tratamento de erros (timeouts, status codes 5xx), mapeamento de dados (payloads de requisição e resposta) e autenticação/autorização (tokens, chaves). Ferramentas como **Postman** (para testes manuais e automatizados simples) e bibliotecas específicas de linguagem, como o **Rest-Assured** (Java) ou o **SuperTest** (Node.js), são essenciais.
- **Exemplo Prático:** Uma aplicação de e-commerce que integra com o serviço de frete dos Correios. O teste de integração deve simular:
 - **Cenário de Sucesso:** Envio de um CEP válido e recebimento do valor do frete e prazo corretos.
 - **Cenário de Falha:** Envio de um CEP inválido e verificação se a aplicação trata o erro (ex: *HTTP status 400 ou 404*), exibindo uma mensagem amigável ao usuário.
 - **Cenário de Timeout:** Simulação de uma falha de conexão com os Correios para validar se a aplicação possui um mecanismo de **retry** ou **circuit breaker** para evitar travamentos.

- **Links de Vídeo:**
 - [Teste da APIs com Requisições POST - AULA #50](#)
 - [Como fazer o planejamento de testes de API Rest](#)

Referência do Slide: Tema 2 - Criação de Cenários Realistas

- **Definição:** É a prática de elaborar **casos de teste que representem as situações mais críticas e frequentes** de uso da integração, garantindo que o sistema se comporte conforme o esperado sob diversas condições.
- **Aprofundamento/Complemento (se necessário):** Cenários realistas envolvem a utilização de dados de teste que sejam representativos dos dados de produção (porém, anonimizados ou *mockados*). Além dos cenários de sucesso e falha óbvios, é fundamental incluir: **testes de segurança** (injeção de SQL, falha de autenticação), **testes de performance** (simulando múltiplas chamadas simultâneas, embora o foco principal seja na Aula 2) e **testes de contrato** (garantindo que o formato da requisição/resposta da API externa não mudou).
- **Exemplo Prático:** Teste de integração com um serviço de pagamento.
 - Cenário: Tentativa de pagamento com um cartão de crédito com saldo insuficiente (API retorna *HTTP status 402 Payment Required*). O sistema deve capturar esse erro e notificar o usuário para tentar outro cartão.
 - Cenário: Teste de idempotência. Enviar a mesma requisição de pagamento duas vezes para garantir que apenas uma transação seja processada pela API externa.
 - **Links de Vídeo:**
 - [Guia para Testes de API Rest \(Playlist\)](#)
 - [Teste Automatizado de API \(Playlist\)](#)

Referência do Slide: Tema 3 - Interpretação de Resultados

- **Definição:** O processo de **análise dos logs, status codes e tempo de resposta** dos testes de integração para validar a confiabilidade e segurança das chamadas feitas a serviços externos.
- **Aprofundamento/Complemento (se necessário):** A interpretação vai além de checar o **status code 200 (OK)**. É preciso verificar o **tempo de latência** (se o serviço externo responde em um tempo aceitável), o **conteúdo do payload de resposta** (se o JSON/XML está no formato esperado) e os **cabeçalhos de segurança** (como *CORS* e *Content-Security-Policy*). Qualquer **status code 4xx** (erro do cliente) ou **5xx** (erro do servidor) deve ser tratado como um potencial ponto de falha que precisa de um tratamento específico na aplicação.
- **Exemplo Prático:** Após a execução de um conjunto de testes, o resultado aponta: **POST /users** retorna **201 Created** (Sucesso), mas a latência média é de **4 segundos**.
 - **Interpretação:** Embora o teste funcional tenha passado, o teste não funcional de performance falhou. 4 segundos é inaceitável para uma criação

de usuário, indicando um gargalo na integração ou na API externa que precisa ser investigado.

- **Links de Vídeo:**

- [Quais são as ferramentas para testes de API Rest mais requisitadas?](#)
- [Teste em Produção é possível? 4 técnicas para fazer testes em produção](#)

Semana 28 - Aula 2

Tópico Principal da Aula: Integração e Testes de Serviços Externos

Subtítulo/Tema Específico: Testes de Carga e Performance

Código da aula: [SIS]ANO2C2B4S28A2

Objetivos da Aula:

- Estruturar e aplicar **testes de carga** em APIs para simular cenários de estresse real[ANO2C2B4S28A2.pdf].
- Configurar simulações de carga baseadas em comportamentos de uso esperado do sistema.
- Entender as métricas essenciais para avaliar a escalabilidade e a estabilidade de APIs em produção.

Recursos Adicionais (Sugestão, pode ser adaptado):

- Caderno para anotações;
- Acesso ao laboratório de informática e/ou internet.
- Ferramentas de Teste de Carga (JMeter, k6).

Exposição do Conteúdo

Referência do Slide: Tema 1 - Estrutura e Aplicação de Testes de Carga

- **Definição: Testes de carga (Load Tests)** avaliam o comportamento do sistema sob uma carga esperada, simulando o número de usuários concorrentes (ou transações por segundo) que a aplicação deve suportar em condições normais de pico.
- **Aprofundamento/Complemento (se necessário):** A estrutura de um teste de carga requer a definição de: **Virtual Users (VUs)** ou **Threads** (usuários virtuais simultâneos), **Ramp-up** (o tempo que leva para atingir o número máximo de VUs) e **Duração** (por quanto tempo a carga máxima será mantida). O **Teste de Estresse (Stress Test)** é um tipo específico de teste de carga onde a capacidade da API é intencionalmente excedida para descobrir o ponto de falha (*breaking point*). O objetivo é garantir que, sob pressão, a API não apenas aguente, mas também falhe de forma controlada e previsível.
- **Exemplo Prático:** Uma API de login deve suportar 500 usuários logando por minuto.

- **Configuração do Teste de Carga (JMeter/k6):** Configurar 500 VUs, com um *ramp-up* de 60 segundos, e duração de 5 minutos, monitorando o tempo de resposta do endpoint `/login`.
- **Links de Vídeo:**
 - [Getting started with API Load Testing \(Stress, Spike, Load, Soak\) - k6](#)
 - [Faça testes de carga em sua Api REST de forma simples com JMeter](#)

Referência do Slide: Tema 2 - Configuração de Simulações

- **Definição:** Consiste em **modelar o teste** para refletir com precisão os diferentes fluxos de trabalho e a proporção de chamadas (*mix de transações*) que os usuários reais fariam na .
- **Aprofundamento/Complemento (se necessário):** Uma simulação eficaz considera que nem todos os usuários estão fazendo a mesma coisa. Se 80% dos usuários apenas leem dados (GET) e 20% criam/atualizam dados (POST/PUT), a simulação deve replicar essa proporção (por exemplo, 80 requisições GET para cada 20 requisições POST). Além disso, os dados de entrada devem ser parametrizados (lidos de um arquivo CSV, por exemplo) para evitar que o cache da API influencie o resultado.
- **Exemplo Prático:** Uma API de Blog.
 - **Mix de Transações:** 70% GET `/posts` (Leitura), 20% GET `/posts/{id}` (Detalhe) e 10% POST `/posts` (Criação).
 - **Simulação:** Em um teste de 1000 requisições, 700 devem ser GET `/posts`, 200 GET `/posts/{id}` com IDs dinâmicos, e 100 POST `/posts` com diferentes payloads de criação de post.
 - **Links de Vídeo:**
 - [JMeter or k6? Your ULTIMATE Guide to Performance Testing Tools!](#)
 - [Performance testing your API with Grafana K6](#)

Referência do Slide: Tema 3 - Métricas Essenciais de Performance

- **Definição:** São os indicadores-chave de desempenho (KPIs) que devem ser monitorados para determinar se a API está funcionando de maneira aceitável sob carga.
- **Aprofundamento/Complemento (se necessário):** As métricas mais críticas incluem:
 - **Latência/Tempo de Resposta:** Tempo que a API leva para retornar a resposta (geralmente medida em milissegundos). É crucial observar a média, o percentil 95 (P95) e o percentil 99 (P99).
 - **Throughput (Vazão):** Número de requisições por segundo (RPS) que a API consegue processar.
 - **Taxa de Erros:** Porcentagem de requisições que retornam status `4xx` ou `5xx`. O ideal é 0% de erros `5xx`.
 - **Utilização de Recursos do Servidor:** CPU, memória, I/O de disco e rede no servidor de aplicação e banco de dados.

- **Exemplo Prático:** Resultados de um Teste de Carga.
 - **Métrica:** Latência P95 (95% das requisições): **1500ms (1.5s)**.
 - **Análise:** O requisito é que a P95 seja no máximo 500ms. O valor obtido indica um gargalo. A equipe deve investigar o que está causando essa lentidão nos 5% das requisições mais lentas.
 - **Links de Vídeo:**
 - [The Best Performance And Load Testing Tool? k6 By Grafana Labs](#)
 - [Como saber se sua API é escalavel? Teste de stress com K6 e Grafana](#)

Semana 28 - Aula 3

Tópico Principal da Aula: Integração e Testes de Serviços Externos

Subtítulo/Tema Específico: Monitoramento de APIs em Produção

Código da aula: [SIS]ANO2C2B4S28A3

Objetivos da Aula:

- Entender a importância do **monitoramento contínuo** para garantir a disponibilidade, desempenho e confiabilidade das.
- Configurar **alertas eficazes** com base em limiares e canais de notificação adequados.
- Utilizar a **análise de logs** para diagnosticar falhas e entender o comportamento da aplicação em tempo real.

Recursos Adicionais (Sugestão, pode ser adaptado):

- Caderno para anotações;
- Acesso ao laboratório de informática e/ou internet.
- Acesso a ferramentas de monitoramento (Prometheus, Grafana, Datadog, etc.).

Exposição do Conteúdo

Referência do Slide: Tema 1 - Ferramentas de Monitoramento e Observabilidade

- **Definição:** O monitoramento de APIs em produção é essencial para garantir **disponibilidade, desempenho e confiabilidade**, permitindo detectar falhas antes que os usuários percebam e fornecendo dados para a tomada de decisões.
- **Aprofundamento/Complemento (se necessário):** O conceito moderno é **Observabilidade**, que se baseia em três pilares: **Métricas** (o que está acontecendo - ex: tempo de CPU, taxa de erro), **Logs** (o que aconteceu - ex: registro de um erro específico) e **Traces** (como a requisição fluiu pelos micros serviços). Ferramentas como **Prometheus** (coleta de métricas *time series*), **Grafana** (criação de *dashboards* para visualização das métricas) e **Datadog/New Relic** (soluções completas de APM - Application Performance Monitoring) são amplamente utilizadas.

- **Exemplo Prático:** Uso do **Prometheus** para coletar a métrica `http_requests_total` e do **Grafana** para exibir em um gráfico o total de requisições por minuto.
 - **Links de Vídeo:**
 - [Dashboard de Monitorando APIs com Grafana | Prometheus](#)
 - [Introdução ao Prometheus e Grafana: Monitoramento inteligente de aplicações](#)

Referência do Slide: Tema 2 - Boas Práticas na Configuração de Alertas

- **Definição:** Configurar alertas é definir que, quando uma métrica ultrapassar um **limiar (threshold)** definido e sustentado por um certo período de tempo, uma **notificação** deve ser disparada para a equipe responsável.
- **Aprofundamento/Complemento (se necessário):** As boas práticas incluem:
 - **Definição de Níveis de Severidade:** Classificar os alertas em **Critical** (o serviço está inoperante), **Alert** (a performance está degradada) e **Warning** (um evento incomum ocorreu, mas o serviço ainda funciona).
 - **Canais de Notificação Adequados:** Usar **SMS/Telefone** para alertas críticos (*Critical*), **Slack/Teams** para alertas de degradação (*Alert*) e **E-mail** para avisos leves (*Warning*).
 - **Evitar Falsos Positivos:** Configurar regras de **repetição ou tempo mínimo** antes do disparo, para que picos momentâneos não gerem alertas desnecessários.
- **Exemplo Prático:** Alerta de Latência.
 - **Regra:** Se a **latência média** do endpoint `/checkout` for superior a **1000ms** por mais de **5 minutos**, dispare um alerta de severidade **Alert** no Slack do time de Back-end.
 - **Links de Vídeo:**
 - [Monitoramento sem surpresas com Prometheus e Grafana](#)
 - [Prometheus e Grafana: Uma forma efetiva de monitorar sistemas](#)

Referência do Slide: Tema 3 - Análise e Utilização de Logs

- **Definição:** Logs são o "raio-x" da aplicação. A análise de logs permite entender **o que, quando, onde e quem** foi afetado por um evento ou falha, sendo crucial para a **detecção de anomalias e debugging** em produção.
- **Aprofundamento/Complemento (se necessário):** Existem diferentes tipos de logs: **Logs de Acesso** (registram requisições HTTP), **Logs de Erro** (registram *exceptions* e falhas) e **Logs de Auditoria** (registram eventos de segurança e ações do usuário). A centralização dos logs (usando o **ELK Stack** - Elasticsearch, Logstash, Kibana - ou soluções *cloud-native*) é fundamental para que o desenvolvedor não precise acessar o servidor manualmente. Deve-se garantir que os logs contenham informações de **contexto** (ID de transação, usuário logado, timestamp).
- **Exemplo Prático:** Um erro ocorre no sistema: `Status code 500 Internal Server Error`.

- **Ação:** O desenvolvedor consulta o sistema de logs centralizado, filtra pelo *timestamp* do erro e encontra o **Log de Erro** que revela: `NullPointerException at UserService.java:150`.
- **Diagnóstico:** O erro foi causado por um objeto nulo no método de serviço, e a informação do log permite que a correção seja feita de forma ágil, sem a necessidade de reproduzir o cenário em ambiente local.
- **Links de Vídeo:**
 - [Grafana: The open and composable observability platform](#) (Link para a plataforma que integra logs, métricas e traces)
 - [Como fazer testes de performance em API Rest? \(Aula 15\)](#) (Apresenta o contexto de performance que gera logs)